

SIMULADOR DE PAGE FAULT

O código implementa uma simulação de gerenciamento de memória por paginação, destacando o comportamento de acesso a páginas e o tratamento de *page faults*. A seguir são explicadas apenas as partes mais relevantes da lógica.

Estrutura da Tabela de Páginas

```
typedef struct{
    int numero_frame;
    char bit_validade;
} Pagina;
```

Essa estrutura é o núcleo do sistema de paginação. O campo `bit_validade` indica se a página está na memória RAM ('v') ou no disco ('i'). Já `numero_frame` informa exatamente onde a página está localizada na RAM. Essa relação é essencial para permitir acessos rápidos.

Inicialização (evita erro crítico)

```
frames[i].num_pagina = -1;
tabela[i].bit_validade = 'i';
```

Essa parte garante que:

- A RAM começa vazia (-1)

- Nenhuma página está carregada ('i')

Sem isso, o programa poderia acessar valores aleatórios da memória (*lixo*), causando comportamentos incorretos.

Carregamento inicial da RAM

```
if(frames[r_frame].num_pagina == -1 && tabela[r_pag].bit_validade == 'i')
```

Aqui ocorre a primeira simulação real do sistema. A condição garante duas coisas importantes:

- Não sobrescrever frames ocupados

- Não carregar páginas duplicadas

Isso mantém a consistência entre RAM e tabela de páginas.

Verificação de HIT (acesso rápido)

```
if(tabela[pag].bit_validade == 'v')
```

Esse é o ponto central da simulação. Quando o bit de validade é 'v', significa que a página já está na RAM. Nesse caso, o acesso é imediato (*hit*), simulando o comportamento ideal de desempenho em sistemas reais.

Tratamento de PAGE FAULT

```
if(frames[j].num_pagina == -1)
```

Se a página não estiver na RAM, ocorre um *page fault*. O sistema então procura um espaço livre na memória para carregar a página.

Ao encontrar:

```
frames[j].num_pagina = pag;  
tabela[pag].bit_validade = 'v';  
tabela[pag].numero_frame = j;
```

A página é carregada e a tabela atualizada. Esse processo simula o carregamento do disco para a RAM.

Caso de memória cheia

```
if(!carregada){  
    cout << "Memoria cheia! Necessario algoritmo de substituicao." << endl;  
}
```

Esse trecho mostra uma limitação proposital do sistema: quando não há espaço disponível, seria necessário aplicar um algoritmo de substituição (como FIFO ou LRU). Isso demonstra entendimento do problema real, mesmo sem implementar a solução.

Geração de acessos aleatórios

```
acessos[i] = rand() % NUM_PAGINAS;
```

Simula requisições de um processo à memória. Isso torna o comportamento do programa dinâmico e mais próximo de um sistema real.

Conclusão (curta e objetiva)

O código demonstra de forma clara os conceitos fundamentais de paginação, como tabela de páginas, frames, *hit* e *page fault*. As partes mais importantes estão

na verificação do bit de validade e no carregamento de páginas na memória, que representam diretamente o funcionamento de sistemas operacionais reais. A ausência de um algoritmo de substituição evidencia uma possível evolução do projeto.